

Tworzenie obiektów w osobnych plikach

Zwróć uwagę na to, że używamy słowa kluczowego **export**

1) Interface + obiekt w osobnym pliku (najczęstsze) (nazwa pliku **user.model.ts** – może być dowolna jednak poniżej wszystkie wyjaśnienia tyczą się podanej nazwy plików)

```
export interface User {  
  id: number;  
  name: string;  
  email: string;  
  roles: string[];  
}
```

teraz na podstawie interfejsu tworzymy sobie tablicę obiektów (w osobnym pliku ja nazwałem go **mock-users.ts**) zauważ, że użyliśmy słowa kluczowego **import**.

Jeśli plik utworzony w miejscu gdzie jest główny plik uruchamiający projekt to pomijasz **../models/**

```
import { User } from '../models/user.model';  
  
export const MOCK_USERS: User[] = [  
  { id: 1, name: 'Ała', email: 'ala@demo.pl', roles: ['admin'] },  
  { id: 2, name: 'Olek', email: 'olek@demo.pl', roles: ['user'] },  
];
```

I teraz w części głównej programu tworzymy nasze obiekty:

```
import { Component } from '@angular/core';
import { MOCK_USERS } from '../data/mock-users';

@Component({
  selector: 'app-users',
  standalone: true,
  template: `
    <ul>
      <li *ngFor="let u of users">{{ u.name }} ({{ u.email }})</li>
    </ul>
  `,
})
export class UsersComponent {
  users = MOCK_USERS; Tutaj mamy tablicę obiektów
}
```

Użyliśmy tutaj dyrektywy ngFor – po to aby z tablicy wyświetlić wszystkie obiekty, a dokładniej pola name i email. O dyrektywach więcej na następnych lekcjach.

2) Klasa (model) w osobnym pliku + tworzenie obiektów przez new

W pliku o nazwie product.model.ts (oczywiście nazwa dowolna ale przykłady odnoszą się do podanych plików)

```
export class Product {
  constructor(
    public id: number,
    public name: string,
    public price: number,
    public vatRate = 0.23
  ) {}

  grossPrice(): number {
    return this.price * (1 + this.vatRate);
  }
}
```

Teraz w nowym pliku **products.ts** tworzymy jak poniżej:
Oczywiście import from bez ”../models/”

```
import { Product } from '../models/product.model';

export const PRODUCTS: Product[] = [
  new Product(1, 'Monitor', 1000),
  new Product(2, 'Klawiatura', 250, 0.23),
];
```

Użycie w pliku głównym programu:

```
import { Component } from '@angular/core';
import { PRODUCTS } from '../data/products';

@Component({
  selector: 'app-products',
  standalone: true,
  template: `
    <div *ngFor="let p of products">
      {{ p.name }}: netto {{ p.price }} / brutto {{ p.grossPrice() | number:'1.2-2' }}
    </div>
  `,
})
export class ProductsComponent {
  products = PRODUCTS;
}
```

Zadania

Zadanie 1: Lista pracowników (proste obiekty)

1. Utwórz folder models i zdefiniuj typ/strukturę obiektu **Employee** (np. id, imię, dział, pensja).
2. Utwórz folder data i w osobnym pliku przygotuj tablicę minimum **8 pracowników** jako stałą eksportowaną.
3. Zaimportuj tablicę

4. W programie głównym, jako funkcja uruchamiająca się po naciśnięciu przycisku, wyświetl ją w tabeli (kolumny: imię, dział, pensja).
5. Dodaj filtrowanie: pokaż tylko pracowników z wybranego działu (np. „IT”).
6. Dodaj podsumowanie: policz i wyświetl **średnią pensję** oraz **liczbę pracowników** po zastosowaniu filtra.

Zadanie 2: Zadania/To-Do (proste obiekty)

1. W folderze models zdefiniuj typ/strukturę obiektu **Task** (np. id, tytuł, priorytet, status).
2. W folderze data utwórz osobny plik z tablicą minimum **10 zadań** jako stałą eksportowaną.
3. Zaimportuj tablicę
4. W programie głównym, jako funkcja uruchamiająca się po naciśnięciu przycisku, wyświetl listę zadań (tytuł, priorytet, status).
5. Dodaj sortowanie: wyświetl zadania od najwyższego priorytetu do najniższego.
6. Dodaj filtrowanie: pokaż tylko zadania o statusie „Do zrobienia” albo tylko „Zrobione”.
7. Dodaj licznik: wyświetl ile jest zadań „Do zrobienia” i ile „Zrobione”.